Individual Technical Report — Member B (Exploration & Statistics)

Introduction to NLP — Assignment 1 (Project Phase 1: Setup, Data Acquisition, Reproducibility & Integration)

Member Name:	Alisha Waseem (Member B — Exploration & Statistics)
Roll Number:	l1f22bscs0180
Team Name:	Khaqan git
Instructor:	Sir Numan
GitHub Account & Branch:	lisha-126 | Branch: feat/alisha-exploration
Repository Link:	https://github.com/Khaqan-J/NLP-Assignment-01-pipeline

1. Scope and Individual Responsibilities

As Member B, I owned Task 2 of the assignment: profiling the raw IMDB 50K corpus before any cleaning, quantifying document-length and class distributions, measuring vocabulary richness and Zipfian structure, and cataloguing the noisy patterns that the cleaning pipeline (Member D) later removes. Concretely, I implemented src/explore.py (corpus statistics, noise prevalence counts, and both diagnostic figures), authored and later rebuilt notebooks/01_exploration.ipynb so that it runs end-to-end with saved outputs, generated the two committed artefacts in reports/figures/, and wrote the exploration section of the shared report (Deliverable B) together with README hygiene fixes. All of my work was delivered through the feature branch feat/alisha-exploration under my own GitHub account, lisha-126.

2. Exploration Methodology and Statistical Profile (Task 2)

The exploration runs on the raw corpus returned by src.data.load_raw_data() twice: once without deduplication, to describe the corpus as delivered, and once with drop_duplicates=True, to report the effective modelling size after removing 418 exact duplicate reviews. Document length is measured by whitespace splitting, which is the neutral baseline before any tokenizer is chosen. Token, type and hapax statistics are computed on a reproducible 5,000-review sample (random_state=42) using the same regex token pattern later adopted in src/tokenization.py (\w+|[^\w\s]), so that exploratory counts stay consistent with the tokenization task. Vocabulary figures use a Counter over all sample tokens: types, type–token ratio (TTR), hapax legomena (types occurring exactly once), and the share of running tokens covered by the 100 most frequent types.

Because the script prints every number it computes and the notebook re-uses the identical function through run_exploration(), the table below is regenerated verbatim on each run and cannot drift from the figures.

Table 1: Verified corpus statistics for the raw IMDB 50K dataset (committed notebook outputs).

Statistic	Verified Value
Documents (raw)	50,000 (no missing values)
Exact duplicates	418 removed → 49,582 unique documents
Words per review (whitespace split)	mean 231.2, median 173, min 4, max 2,470
Target distribution	25,000 positive / 25,000 negative (perfectly balanced)
Tokens in the 5,000-review sample	1,473,795
Types in the sample	47,046 (type–token ratio 0.032)
Hapax legomena	19,951 (42.4% of all types)
Token share of the top 100 types	57.0%

Two consequences follow for Phase 2. First, the classes are exactly balanced, so accuracy is a meaningful headline metric, although macro-F1 should still be reported. Second, 42.4% of types occur exactly once while 100 types

cover 57.0% of running tokens: a min-frequency threshold and a bounded vocabulary are necessary, and rare types carry almost no usable signal. The right-skewed length distribution (median 173 versus mean 231.2, tail to 2,470 words) also means raw token counts will be dominated by long reviews unless sequence length is capped or normalised.

3. Messy-Pattern Catalogue and Real Evidence

I catalogued seven noise categories with vectorised regular expressions evaluated over all 50,000 reviews, so the numbers describe the whole corpus rather than the sample. Each row of Table 2 corresponds to a pattern that the cleaning pipeline must neutralise, which is why these counts, not just the figures, form part of the Task 2 deliverable.

Table 2: Prevalence of messy patterns across all 50,000 raw reviews.

Noise category	Reviews	Share	Pattern basis
HTML line breaks ( )	29,200	58.4%	<br\s*/?>
ALL-CAPS words of 4+ letters	11,339	22.7%	\b[A-Z]{4,}\b
Non-ASCII characters	4,660	9.3%	[^\x00-\x7F]
Explicit rating (8/10, 3 out of 10)	4,381	8.8%	digits + 10-scale
Extra spaces / space before punctuation	3,196	6.4%	+ | \s+[,.!?]
Elongated words	1,539	3.1%	([A-Za-z])\1{2,}
URLs	202	0.4%	https?:// | www\.

The single most important finding is that 58.4% of reviews carry raw HTML line breaks. Because those tags are glued to words in the source text (for example Seriously.

), any tokenizer applied before tag removal produces junk tokens such as /><br, which is why HTML stripping is step 2 of the cleaning order. Representative excerpts follow, quoted verbatim from the corpus.

Table 3: Real messy excerpts from the raw corpus (row indices are the pandas index of the raw CSV).

Row	Pattern shown	Excerpt (truncated)
0	Raw HTML tags	...exactly what happened with me. The first thing that struck me about Oz...
3	ALL-CAPS + explicit rating	...I expected to see a BOOGEYMAN similar movie... / ...3 out of 10 just for the well playing parents & descent dialogs...
23	Character elongation	...the movie goes WWWAAAAAYYYYY downhill. The animation is VERY b...

25	Space before punctuation	...She is a good actress , but plays a very naive...
29	Non-ASCII characters	...so many times that clichéd dialogue, rehashed plot...
—	Slash-form rating	Direction = 9/10 ... Total = 9/10 (captured by the rating pattern, 4,381 reviews)

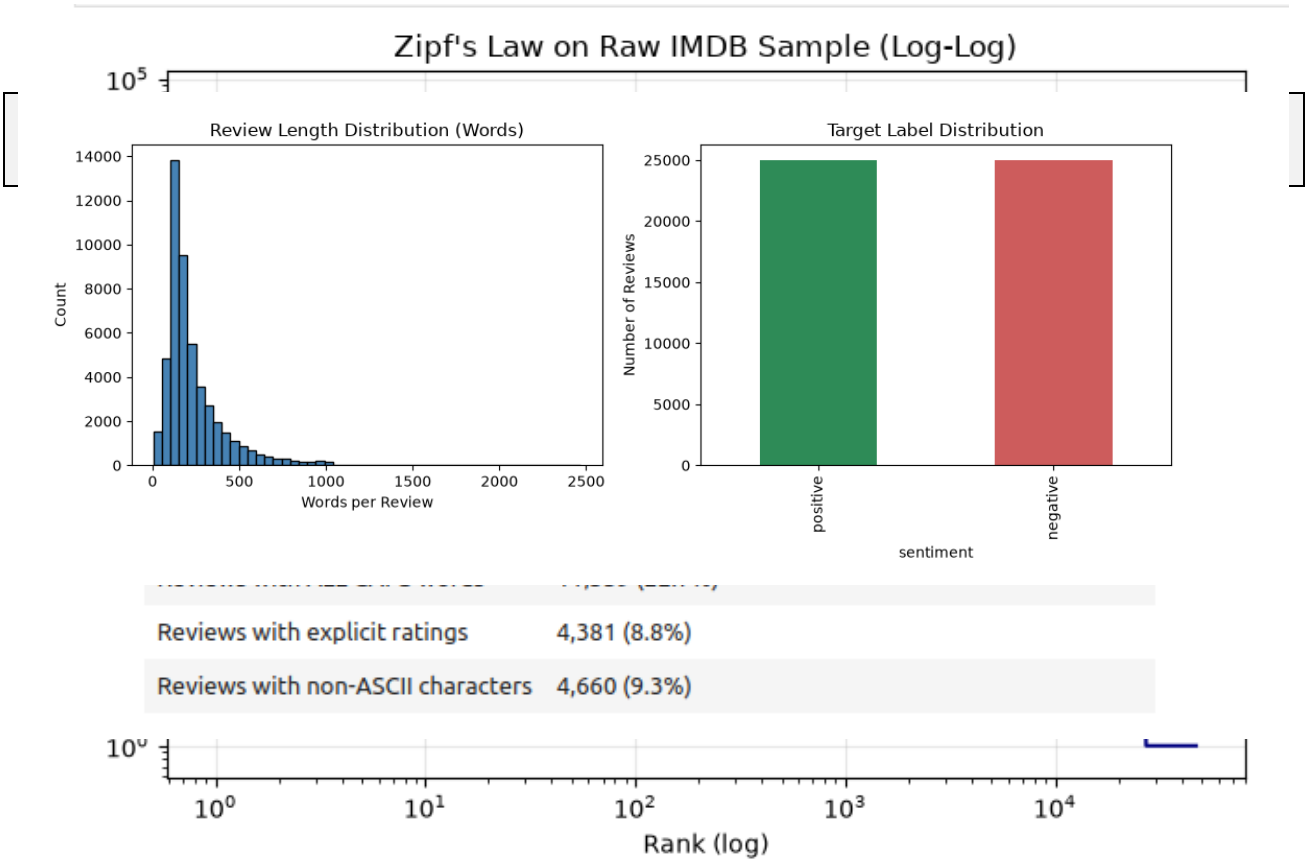
4. Visual Diagnostics and Committed Artefacts

Both figures are produced by src/explore.py through src.config.FIGURES_DIR, so the command-line run and the notebook write to the same paths and cannot diverge. Until this task, reports/figures/ contained only .gitkeep: the plots existed on disk but were never committed, so a marker cloning the repository had no visual evidence of the exploration. The two PNG files are now versioned alongside the notebook that displays them.

Table 4: Committed figure artefacts.

Artefact	Dimensions	Size	Content
reports/figures/zipf_raw.png	1050 x 750 px (150 dpi)	50,665 B	Log-log rank–frequency curve for the 5,000-review sample
reports/figures/length_and_labels.png	1650 x 675 px (150 dpi)	54,286 B	Review-length histogram beside the target-label bar chart

On the log-log plot, rank against frequency follows the near-linear descent that Zipf's law predicts, with a steep head for function words and punctuation, a linear middle, and a characteristic cliff at frequency 1 where the 19,951 hapax legomena sit. The second figure shows the right-skewed length distribution described in Table 1 next to a perfectly balanced 25,000 / 25,000 label split, which is the empirical justification for using accuracy as a primary Phase 2 metric.



5. Notebook Execution Defect and Resolution

During integration verification, the team's audit found that notebooks/01_exploration.ipynb contained three Markdown cells and zero executable code cells: the Python had been pasted as prose, so the notebook could not run and displayed no outputs. Since the assignment requires an executed notebook as evidence, I took ownership of the defect on my own branch rather than patching it on main, and rebuilt the notebook around the existing, unchanged run_exploration() function.

Table 5: Defect, root cause and resolution for the exploration notebook.

Defect	Affected file	Engineering root cause	Resolution implemented
Markdown-only notebook	notebooks/01_exploration.ipynb	Code was pasted as Markdown: 3 cells, 0 executable cells, no outputs	Rebuilt as 9 code + 11 Markdown cells, executed end-to-end
Missing figure artefacts	reports/figures/	Directory held only .gitkeep; plots were generated but never committed	Committed zipf_raw.png and length_and_labels.png with the notebook
Path portability	notebook cells	Absolute sandbox paths would break on a marker's machine	sys.path append for src imports; FIGURES_DIR resolved from src.config
Regex capture-group warning	noise-prevalence cell	Series.str.contains with grouping patterns raises pandas UserWarning	Pre-compiled patterns applied with Series.apply

The rebuilt notebook imports src.data.load_raw_data() (no hard-coded CSV path), reproduces every number in Table 1 and Table 2, and embeds both figures inline. Execution with jupyter nbconvert --execute completed with exit status 0: 20 cells, 8 cells carrying outputs, 2 embedded PNG figures, 0 error outputs, and no sandbox-specific paths in the saved file (119,606 bytes). The commit also restores the notebook's kernel-independent behaviour, so a fresh clone reproduces the outputs with:

```
git clone https://github.com/Khaqan-J/NLP-Assignment-01-pipeline.git
cd NLP-Assignment-01-pipeline
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.data          # downloads the 50K CSV
python -m src.explore       # rewrites both figures in reports/figures/
jupyter nbconvert --execute --inplace notebooks/01_exploration.ipynb
```

Out[5]:

	Metric	Value
0	Reviews sampled	5,000
1	Tokens	1,473,795
2	Types (unique tokens)	47,046
3	Type-token ratio (TTR)	0.032
4	Hapax legomena (seen once)	19,951
5	Hapax share of types	42.4%
6	Share of tokens in top 100 types	57.0%

In [6]:
```
top_tokens = pd.DataFrame(counts.most_common(10), columns=["Token", "Frequency"])
top_tokens
```

Out[6]:

	Token	Frequency
0	.	65358
1	the	57986
2	,	55035
3	a	31364
4	and	31182
5	of	28900
6	'	26805
7	to	26618
8	/	21512
9	is	21138

6. Documentation Contribution

Beyond the code artefacts I maintained the project's front door. In PR #4 I removed stray citation markers left over from drafting and repaired broken relative links in README.md, so that the dataset, module and reproducibility sections resolve correctly from a fresh clone. In PR #7 I added the exploration-findings section to README.md, summarising the corpus size, length distribution, vocabulary richness and noise prevalence with the same numbers that src/explore.py prints, which keeps the README and the code consistent. Both changes were raised as separate, reviewable pull requests rather than folded into the feature branch, which kept the review diff of each change small.

7. Branch Contents and Commit-Level History

The feature branch feat/alisha-exploration holds the complete Task 2 implementation. Table 6 lists every commit behind those deliverables with the exact files and line counts each one changed, and Table 7 summarises what the branch contains once those commits are applied, so the branch can be audited without opening a single diff.

Table 6: Commit-level history of the Task 2 deliverables (files, additions and deletions per commit).

Commit	Date (UTC)	Files changed	Content and engineering effect
d4e6ca0	7 Nov 17:01	2 files, +140 / -0	src/explore.py (new, 92 lines): statistics, seven noise patterns, Zipf and length/label plot functions; notebooks/01_exploration.ipynb (new, 48 lines, Markdown-only, no executable cells). Authored under the Team Lead account: see Section 8.
47b5b24	7 Nov 19:59	README.md, +11 / -11	Removed stray [cite: ...] markers from the dataset and team tables and repaired broken relative links so every section resolves from a fresh clone.
d20a22d	7 Nov 20:22	README.md, +16 / -1	Added the exploration-findings section: corpus size, length statistics, vocabulary richness and noise prevalence, matching the numbers src/explore.py prints.
b52bfbc	11 Nov 16:20	3 files, +948 / -13	notebooks/01_exploration.ipynb rebuilt (+948 / -13) into 9 executable code cells with 8 outputs, 2 inline figures and 0 errors; reports/figures/zipf_raw.png (new, 50,665 B) and reports/figures/length_and_labels.png (new, 54,286 B) committed.

Table 7: Deliverable files present on feat/alisha-exploration after commit b52bfbc.

File	Size	Role in the deliverable
src/explore.py	92 lines	Single source of truth: loads the raw corpus, prints every statistic, counts the seven noise categories and writes both figures
notebooks/01_exploration.ipynb	119,606 B	Executed evidence notebook: 20 cells (9 code, 11 Markdown), 8 cells with outputs, 2 inline plots, 0 error outputs
reports/figures/zipf_raw.png	50,665 B	Zipf log-log rank-frequency curve for the 5,000-review sample
reports/figures/length_and_labels.png	54,286 B	Review-length histogram beside the balanced target-label bar chart

Because the notebook imports run_exploration() instead of duplicating logic, the branch carries exactly one implementation of each statistic: a change to src/explore.py propagates to the notebook, the console output and the committed figures in the same run, which is why the numbers in Tables 1 and 2 cannot disagree with the plotted artefacts.

8. Git Workflow Narrative and Engineering Retrospective

All of my work lives on the branch feat/alisha-exploration and was integrated into main through reviewed pull requests opened from my own account, lisha-126. One historical defect is recorded honestly: the original Task 2 implementation (src/explore.py, the statistics script, both plot functions and the first notebook) was committed as d4e6ca0 and opened as PR #2 through the Team Lead's account (Khaqan-J) before branch protection existed on main. In line with Git integrity rules we did not rebase, amend or rewrite that commit; instead every subsequent change of mine was authored, committed and pushed from lisha-126, and branch protection requiring review was enabled afterwards so the situation cannot recur.

Table 8: My commits and pull requests, with the account that authored each one.

Commit / PR	Branch	Author account	Contribution
d4e6ca0 (7 Nov)	feat/alisha-exploration	Khaqan-J (historical defect)	src/explore.py, corpus statistics, Zipf and length/label plot functions, first notebook
47b5b24 / PR #4 (7 Nov, merged 20:13)	feat/alisha-readme-fix	lisha-126	README: removed stray citation markers, fixed broken links
d20a22d / PR #7 (7 Nov, merged 20:37)	feat/alisha-readme-exploration	lisha-126	README: added exploration findings section with the Task 2 numbers
b52bfbc / PR #9 (11 Nov, merged 17:15 via 8eb1bd2)	feat/alisha-exploration	lisha-126	Executed notebook (20 cells, 0 errors, 2 inline plots) plus both figure PNGs; 3 files, +948 / -13

Early in the project my exploratory work also passed through the shared pipeline: the exploration numbers I documented were reused by Member D to justify the cleaning order, and the HTML and elongation findings in Table 2 directly motivated steps 2 and 5 of the 8-step normalisation. Cross-checking my statistics against the cleaning pipeline's before/after vocabulary report is what surfaced the notebook defect in Section 5.

Each pattern below is counted over all 50,000 raw reviews with a single regex pass. These are the defects the cleaning stage must handle.

In [8]:

```
reviews = df_raw["review"]
mess_checks = {
    "HTML line breaks (<br />)": re.compile(r"<br\s*/?>", re.IGNORECASE),
    "ALL-CAPS words of 4+ letters": re.compile(r"\b[A-Z]{4,}\b"),
    "Explicit rating (8/10, 3 out of 10)": re.compile(
        r"\b\d+(?:\.\d+)?\s*(?:/\s*10|out of 10)\b", re.IGNORECASE
    ),
    "Non-ASCII characters": re.compile(r"[^\x00-\x7F]"),
    "Extra spaces or space before punct": re.compile(r"  +|\s+[,.!?]"),
    "Elongated words": re.compile(r"([A-Za-z])\1{2,}"),
    "URLs": re.compile(r"https?://|www\.", re.IGNORECASE),
}

rows = []
for name, pattern in mess_checks.items():
    count = int(reviews.apply(lambda text: pattern.search(text) is not None).sum())
    rows.append((name, count, count / len(df_raw) * 100))

mess_table = pd.DataFrame(rows, columns=["Pattern", "Reviews", "Share (%)"])
mess_table["Share (%)"] = mess_table["Share (%)"].round(1)
mess_table
```

Out[8]:

	Pattern	Reviews	Share (%)
0	HTML line breaks ( )	29200	58.4
1	ALL-CAPS words of 4+ letters	11339	22.7
2	Explicit rating (8/10, 3 out of 10)	4381	8.8
3	Non-ASCII characters	4660	9.3
4	Extra spaces or space before punct	3196	6.4
5	Elongated words	1539	3.1
6	URLs	202	0.4

What I would change next: (1) open the feature branch from my own account before writing the first line of code, so that authorship never depends on who pushes the initial commit; (2) execute notebooks before the first commit rather than after review, since a Markdown-only notebook is invisible to a reviewer reading the rendered diff; and (3) assert the headline statistics in a small test so that a later change to tokenization or sampling cannot silently alter the numbers quoted in the report.